

Spring Framework

Complete Learning Guide

Spring Core • Spring AOP • Spring JDBC

Spring JPA • Eclipse & Maven

From Scratch – Definitions, Usage & Examples

Table of Contents

1. Eclipse IDE & Maven

- 1.1 Eclipse IDE Overview
- 1.2 Installing Eclipse
- 1.3 Maven Fundamentals
- 1.4 pom.xml Deep-Dive
- 1.5 Maven Lifecycle

2. Spring Core

- 2.1 IoC & Dependency Injection
- 2.2 Bean Lifecycle
- 2.3 Bean Scopes
- 2.4 Spring Annotations
- 2.5 ApplicationContext
- 2.6 Spring Configuration Styles

3. Spring AOP

- 3.1 AOP Concepts
- 3.2 Advice Types
- 3.3 Pointcut Expressions
- 3.4 @Aspect Annotations
- 3.5 AOP Proxy Mechanism

4. Spring JDBC

- 4.1 JdbcTemplate
- 4.2 NamedParameterJdbcTemplate
- 4.3 RowMapper
- 4.4 Transaction Management
- 4.5 DataSource Configuration

5. Spring JPA / Spring Data JPA

- 5.1 JPA Fundamentals
- 5.2 Entity Mapping
- 5.3 Spring Data Repository
- 5.4 JPQL & Native Queries
- 5.5 Relationships
- 5.6 Transactions

CHAPTER 1

Eclipse IDE & Maven

Setting up the development environment for Spring projects

1.1 Eclipse IDE – Overview

What is Eclipse?

Eclipse is a free, open-source Integrated Development Environment (IDE) primarily used for Java development. It provides a rich set of plugins, a powerful editor, debugger, and build integration. The Spring Tool Suite (STS) is a customised Eclipse distribution pre-loaded with Spring-specific plugins.

Key Eclipse concepts:

- **Workspace** – A directory on disk that holds your projects and IDE settings.
- **Project** – A Java project containing source files, configurations, and dependencies.
- **Perspective** – A pre-defined layout of views (Java perspective, Debug perspective, etc.).
- **View** – A panel inside Eclipse (Package Explorer, Console, Problems, Outline).
- **Editor** – The central panel where you write and edit code.
- **Plugin** – An extension that adds new features (e.g., Spring Tools 4 plugin).

1.2 Installing Eclipse & Spring Tools

- Download **Eclipse IDE for Enterprise Java Developers** from eclipse.org/downloads.
- Alternatively, download **Spring Tool Suite 4 (STS4)** from spring.io/tools – it bundles Spring Tools.
- Install **JDK 17+** (Amazon Corretto, Temurin, or Oracle JDK) and set `JAVA_HOME`.
- In Eclipse: Help → Eclipse Marketplace → search *Spring Tools 4* → Install.

1.3 Maven – Fundamentals

What is Maven?

Apache Maven is a build automation and project management tool. It uses a declarative XML file called `pom.xml` (Project Object Model) to describe project structure, dependencies, build plugins, and lifecycle phases. Maven downloads dependencies from the Maven Central Repository automatically.

Core Maven concepts:

- **POM (`pom.xml`)** – The heart of a Maven project. Defines metadata, dependencies, plugins.
- **GroupId** – A unique organisation identifier, e.g. *com.example*.
- **ArtifactId** – The project/module name, e.g. *spring-demo*.

- **Version** – The release version, e.g. *1.0.0*.
- **Repository** – Where Maven downloads JARs from (local ~/.m2, remote Central).
- **Dependency** – An external library your project uses.
- **Plugin** – A tool that performs build tasks (compile, test, package).

1.4 The pom.xml File

■ pom.xml – Basic Spring Project

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0">
  <modelVersion>4.0.0</modelVersion>

  <groupId>com.example</groupId>
  <artifactId>spring-demo</artifactId>
  <version>1.0.0</version>
  <packaging>jar</packaging>

  <properties>
    <java.version>17</java.version>
    <spring.version>6.1.0</spring.version>
  </properties>

  <dependencies>
    <!-- Spring Context (Core + Beans + AOP + Expression) -->
    <dependency>
      <groupId>org.springframework</groupId>
      <artifactId>spring-context</artifactId>
      <version>${spring.version}</version>
    </dependency>
    <!-- H2 In-Memory Database for testing -->
    <dependency>
      <groupId>com.h2database</groupId>
      <artifactId>h2</artifactId>
      <version>2.2.224</version>
    </dependency>
  </dependencies>

  <build>
    <plugins>
      <plugin>
        <groupId>org.apache.maven.plugins</groupId>
        <artifactId>maven-compiler-plugin</artifactId>
        <configuration>
          <source>17</source>
          <target>17</target>
```

```
</configuration>
</plugin>
</plugins>
</build>
</project>
```

1.5 Maven Build Lifecycle

Phase	Description	Command
validate	Validates project structure	mvn validate
compile	Compiles source code	mvn compile
test	Runs unit tests	mvn test
package	Packages compiled code (JAR/WAR)	mvn package
verify	Runs integration tests	mvn verify
install	Installs artifact to local repository	mvn install
deploy	Deploys to remote repository	mvn deploy
clean	Removes target/ directory	mvn clean

CHAPTER 2

Spring Core

Inversion of Control, Dependency Injection, Bean Lifecycle & Configuration

2.1 Inversion of Control (IoC) & Dependency Injection (DI)

Inversion of Control (IoC)

IoC is a design principle where the control of object creation and lifecycle management is transferred from the application code to a framework (the Spring IoC Container). Instead of your code saying 'new Service()', the container creates and wires the objects.

Dependency Injection (DI)

DI is the mechanism Spring uses to implement IoC. The container injects the dependencies (collaborating objects) that a class needs, rather than the class creating them itself. This promotes loose coupling and testability.

Types of Dependency Injection:

- **Constructor Injection** – Dependencies passed via constructor (recommended, supports immutability).
- **Setter Injection** – Dependencies set via JavaBean setter methods (optional dependencies).
- **Field Injection** – Dependencies injected directly into fields via `@Autowired` (not recommended for production).

Constructor Injection Example:

■ Constructor Injection

```
// Service interface
public interface GreetingService {
    String greet(String name);
}

// Service implementation
@Service
public class GreetingServiceImpl implements GreetingService {
    @Override
    public String greet(String name) {
        return "Hello, " + name + "!";
    }
}

// Client class - dependencies injected via constructor
@Component
```

```

public class AppController {
    private final GreetingService greetingService;

    // Spring detects single constructor and auto-injects
    public AppController(GreetingService greetingService) {
        this.greetingService = greetingService;
    }

    public void run() {
        System.out.println(greetingService.greet("World"));
    }
}

```

Setter Injection Example:

■ Setter Injection

```

@Component
public class ReportService {
    private EmailService emailService;

    @Autowired // Spring calls this setter to inject
    public void setEmailService(EmailService emailService) {
        this.emailService = emailService;
    }
}

```

2.2 Spring IoC Container & ApplicationContext

ApplicationContext

The ApplicationContext is the central interface to the Spring IoC container. It is responsible for instantiating, configuring, and assembling beans. Common implementations: AnnotationConfigApplicationContext (Java config), ClassPathXmlApplicationContext (XML config), and GenericWebApplicationContext (web applications).

■ Bootstrapping the ApplicationContext

```

// Java-based configuration
@Configuration
@ComponentScan("com.example") // scan this package for @Component classes
public class AppConfig {
    @Bean
    public DataSource dataSource() {
        DriverManagerDataSource ds = new DriverManagerDataSource();
        ds.setDriverClassName("org.h2.Driver");
        ds.setUrl("jdbc:h2:mem:testdb");
        return ds;
    }
}

```

```
// Bootstrap the container
public class Main {
    public static void main(String[] args) {
        ApplicationContext ctx =
            new AnnotationConfigApplicationContext(AppConfig.class);

        AppController controller = ctx.getBean(AppController.class);
        controller.run();
    }
}
```

2.3 Bean Lifecycle

Spring Bean

A Spring Bean is any object that is instantiated, assembled, and managed by the Spring IoC container. Beans are defined via `@Component`, `@Service`, `@Repository`, `@Controller` annotations or via `@Bean` methods inside `@Configuration` classes.

Bean Lifecycle Phases:

- **1. Instantiation** – Spring creates the bean instance (calls constructor).
- **2. Populate Properties** – Spring injects dependencies (DI happens here).
- **3. BeanNameAware / BeanFactoryAware** – Spring calls awareness interfaces.
- **4. BeanPostProcessor (before)** – Pre-initialisation processing.
- **5. @PostConstruct / InitializingBean.afterPropertiesSet()** – Custom init logic.
- **6. BeanPostProcessor (after)** – Post-initialisation processing.
- **7. Bean is ready for use** – Application uses the bean.
- **8. @PreDestroy / DisposableBean.destroy()** – Cleanup before container shuts down.

■ @PostConstruct & @PreDestroy

```
@Component
public class DatabasePool {
    @PostConstruct
    public void init() {
        System.out.println("Pool initialised - open connections");
    }

    @PreDestroy
    public void cleanup() {
        System.out.println("Pool destroyed - close connections");
    }
}
```

2.4 Bean Scopes

Scope	Description	Usage
singleton	One instance per Spring container (default)	@Scope("singleton")
prototype	New instance every time bean is requested	@Scope("prototype")
request	One instance per HTTP request (web only)	@RequestScope
session	One instance per HTTP session (web only)	@SessionScope
application	One instance per ServletContext (web only)	@ApplicationScope

■ Prototype Scope

```
@Component
@Scope("prototype") // new instance each time
public class ShoppingCart {
    private List<String> items = new ArrayList<>();
    public void addItem(String item) { items.add(item); }
}
```

2.5 Key Spring Annotations

Annotation	Purpose
@Component	Generic stereotype; marks a class as Spring-managed bean
@Service	Marks service-layer classes (business logic)
@Repository	Marks DAO/persistence-layer classes; enables exception translation
@Controller	Marks MVC controller classes
@RestController	@Controller + @ResponseBody; for REST APIs
@Configuration	Marks a class as a source of bean definitions
@Bean	Declares a method that produces a Spring-managed bean
@Autowired	Injects a dependency by type
@Qualifier("x")	Disambiguates when multiple beans of the same type exist
@Value("\${k}")	Injects a property value from application.properties
@ComponentScan	Configures packages to scan for components
@Lazy	Delays bean creation until first use
@Primary	Marks a bean as the default candidate for injection

@Value & @PropertySource Example:

■ @Value injection from properties file

```
# application.properties
app.name=SpringDemo
```

```
app.timeout=30

@Configuration
@PropertySource("classpath:application.properties")
public class AppConfig { }

@Service
public class AppInfo {
    @Value("${app.name}")
    private String name;

    @Value("${app.timeout}")
    private int timeout;

    public void print() {
        System.out.println(name + " timeout=" + timeout);
    }
}
```

CHAPTER 3

Spring AOP

Aspect-Oriented Programming – Cross-Cutting Concerns Made Easy

3.1 AOP Concepts

What is AOP?

Aspect-Oriented Programming (AOP) is a programming paradigm that separates cross-cutting concerns (logging, security, transactions, caching) from core business logic. Instead of scattering the same code across many classes, AOP centralises it in an Aspect.

Core AOP Terminology:

Term	Definition	Example
Aspect	A module encapsulating a cross-cutting concern	LoggingAspect class
Join Point	A point in execution where advice CAN be applied	Method execution
Advice	The action taken at a Join Point	@Before, @After, @Around
Pointcut	An expression that selects specific Join Points	execution(* com.example.*(..))
Target	The object whose method is being advised	OrderService
Proxy	AOP-wrapped object that intercepts calls	JDK/CGLIB proxy
Weaving	Linking aspects with application code	Done at runtime by Spring

3.2 Advice Types

Advice	When It Runs	Annotation
Before	Before the method executes	@Before
After Returning	After method returns successfully	@AfterReturning
After Throwing	After method throws an exception	@AfterThrowing
After (Finally)	After method regardless of outcome	@After
Around	Wraps the method; most powerful; can suppress call	@Around

3.3 Complete AOP Example

■ Target Service

```
// 1. Add dependency in pom.xml
// spring-aspects + aspectjweaver

// 2. Enable AOP in configuration
@Configuration
@EnableAspectJAutoProxy // enables @AspectJ support
@ComponentScan("com.example")
public class AppConfig { }

// 3. Target Service
@Service
public class OrderService {
    public String placeOrder(String item) {
        System.out.println("Placing order: " + item);
        return "ORDER-" + item.toUpperCase();
    }

    public void cancelOrder(String id) {
        throw new RuntimeException("Order not found: " + id);
    }
}
```

■ LoggingAspect – All Advice Types

```
// 4. Aspect class
@Aspect
@Component
public class LoggingAspect {

    // Pointcut – matches all methods in com.example.service package
    @Pointcut("execution(* com.example.service.*(..))")
    public void serviceMethods() {}

    // @Before – runs BEFORE the method
    @Before("serviceMethods()")
    public void logBefore(JoinPoint jp) {
        System.out.println("[BEFORE] " + jp.getSignature().getName());
    }

    // @AfterReturning – captures return value
    @AfterReturning(pointcut = "serviceMethods()", returning = "result")
    public void logAfterReturn(JoinPoint jp, Object result) {
        System.out.println("[RETURN] " + jp.getSignature().getName()
            + " returned: " + result);
    }
}
```

```

// @AfterThrowing - captures exception
@AfterThrowing(pointcut = "serviceMethods()", throwing = "ex")
public void logException(JoinPoint jp, Exception ex) {
    System.out.println("[ERROR] " + jp.getSignature().getName()
+ " threw: " + ex.getMessage());
}

// @Around - full control; must call proceed()
@Around("execution(* com.example.service.OrderService.placeOrder(..)")
public Object timeMethod(ProceedingJoinPoint pjp) throws Throwable {
    long start = System.currentTimeMillis();
    Object result = pjp.proceed(); // execute the actual method
    long elapsed = System.currentTimeMillis() - start;
    System.out.println("[TIMER] placeOrder took " + elapsed + "ms");
    return result;
}
}

```

3.4 Pointcut Expression Syntax

- **execution(*.*(..))** – Any method, any class, any args.
- **execution(public String com.example.service.*.get*(String))** – Public String-returning getXxx methods in service package with a String arg.
- **within(com.example.service.*)** – All methods in the service package.
- **@annotation(org.springframework.transaction.annotation.Transactional)** – Methods annotated with @Transactional.
- **bean(orderService)** – All methods on the bean named "orderService".
- **args(java.lang.String)** – Methods that take a single String argument.

CHAPTER 4

Spring JDBC

Simplified Database Access with JdbcTemplate

4.1 Overview & Setup

What is Spring JDBC?

Spring JDBC provides a clean, template-based abstraction over the raw JDBC API. It eliminates boilerplate code (opening connections, creating statements, handling ResultSets, closing resources) while keeping full control of your SQL queries. The central class is JdbcTemplate.

■ Maven Dependencies

```
<!-- pom.xml dependencies -->
<dependency>
<groupId>org.springframework</groupId>
<artifactId>spring-jdbc</artifactId>
<version>6.1.0</version>
</dependency>
<dependency>
<groupId>com.h2database</groupId>
<artifactId>h2</artifactId>
<version>2.2.224</version>
</dependency>
```

4.2 JdbcTemplate – Core Operations

■ JdbcTemplate Configuration

```
// Configuration
@Configuration
public class DbConfig {
    @Bean
    public DataSource dataSource() {
        DriverManagerDataSource ds = new DriverManagerDataSource();
        ds.setDriverClassName("org.h2.Driver");
        ds.setUrl("jdbc:h2:mem:testdb;DB_CLOSE_DELAY=-1");
        ds.setUsername("sa"); ds.setPassword("");
        return ds;
    }
    @Bean
```

```

public JdbcTemplate jdbcTemplate(dataSource ds) {
    return new JdbcTemplate(ds);
}
}

```

■ EmployeeDao – Full CRUD with JdbcTemplate

```

// Entity
public class Employee {
    private int id; private String name; private double salary;
    // constructors, getters, setters ...
}

@Repository
public class EmployeeDao {
    private final JdbcTemplate jdbc;

    public EmployeeDao(JdbcTemplate jdbc) { this.jdbc = jdbc; }

    // DDL
    public void createTable() {
        jdbc.execute("CREATE TABLE IF NOT EXISTS employee " +
            "(id INT PRIMARY KEY, name VARCHAR(50), salary DOUBLE)");
    }

    // INSERT
    public int save(Employee e) {
        return jdbc.update("INSERT INTO employee VALUES (?, ?, ?)",
            e.getId(), e.getName(), e.getSalary());
    }

    // UPDATE
    public int update(Employee e) {
        return jdbc.update("UPDATE employee SET name=?, salary=? WHERE id=?",
            e.getName(), e.getSalary(), e.getId());
    }

    // DELETE
    public int delete(int id) {
        return jdbc.update("DELETE FROM employee WHERE id=?", id);
    }

    // SELECT single row
    public Employee findById(int id) {
        return jdbc.queryForObject(
            "SELECT * FROM employee WHERE id=?",
            (rs, rowNum) -> new Employee(
                rs.getInt("id"),

```

```

rs.getString("name"),
rs.getDouble("salary")
), id);
}

// SELECT all rows
public List<Employee> findAll() {
return jdbc.query(
"SELECT * FROM employee",
(rs, rowNum) -> new Employee(
rs.getInt("id"),
rs.getString("name"),
rs.getDouble("salary")
));
}

// Scalar query
public int count() {
return jdbc.queryForObject("SELECT COUNT(*) FROM employee", Integer.class);
}
}

```

4.3 RowMapper

RowMapper

RowMapper is a functional interface that maps each row of a ResultSet to a Java object. It is reusable and keeps the mapping logic separate from your DAO methods.

■ Custom RowMapper

```

// Reusable RowMapper
public class EmployeeRowMapper implements RowMapper<Employee> {
@Override
public Employee mapRow(ResultSet rs, int rowNum) throws SQLException {
Employee e = new Employee();
e.setId(rs.getInt("id"));
e.setName(rs.getString("name"));
e.setSalary(rs.getDouble("salary"));
return e;
}
}

// Usage in DAO
public List<Employee> findBySalaryAbove(double min) {
return jdbc.query(
"SELECT * FROM employee WHERE salary > ?",

```



```
new EmployeeRowMapper(), min);  
}
```

4.4 NamedParameterJdbcTemplate

NamedParameterJdbcTemplate

Instead of positional ? placeholders, NamedParameterJdbcTemplate lets you use named parameters prefixed with : (e.g. :name), making complex queries more readable.

■ NamedParameterJdbcTemplate Usage

```
@Repository  
public class ProductDao {  
    private final NamedParameterJdbcTemplate namedJdbc;  
  
    public ProductDao(DataSource ds) {  
        this.namedJdbc = new NamedParameterJdbcTemplate(ds);  
    }  
  
    public int save(String name, double price) {  
        String sql = "INSERT INTO product (name, price) VALUES (:name, :price)";  
        Map<String, Object> params = new HashMap<>();  
        params.put("name", name);  
        params.put("price", price);  
        return namedJdbc.update(sql, params);  
    }  
  
    // BeanPropertySqlParameterSource maps bean properties to named params  
    public int saveProduct(Product p) {  
        String sql = "INSERT INTO product (name, price) VALUES (:name, :price)";  
        return namedJdbc.update(sql, new BeanPropertySqlParameterSource(p));  
    }  
}
```

4.5 Transaction Management

@Transactional

Spring's declarative transaction management wraps a method (or class) in a database transaction. If the method completes successfully the transaction is committed; if an unchecked exception is thrown it is rolled back automatically.

■ Declarative Transaction with @Transactional

```
@Configuration  
@EnableTransactionManagement // activates @Transactional support  
public class DbConfig {  
    @Bean  
    public PlatformTransactionManager transactionManager(DataSource ds) {  
        return new DataSourceTransactionManager(ds);  
    }  
}
```

```
}  
}  
  
@Service  
public class TransferService {  
    private final EmployeeDao dao;  
    public TransferService(EmployeeDao dao) { this.dao = dao; }  
  
    @Transactional // commit on success, rollback on RuntimeException  
    public void transfer(int fromId, int toId, double amount) {  
        Employee from = dao.findById(fromId);  
        Employee to = dao.findById(toId);  
        from.setSalary(from.getSalary() - amount);  
        to.setSalary(to.getSalary() + amount);  
        dao.update(from);  
        dao.update(to);  
        // If an exception occurs here, both updates are rolled back  
    }  
}
```

CHAPTER 5

Spring Data JPA

Object-Relational Mapping with JPA, Hibernate & Spring Data Repositories

5.1 JPA Fundamentals

What is JPA?

The Java Persistence API (JPA) is a Java specification for Object-Relational Mapping (ORM). It defines how Java objects (Entities) map to database tables and how to query them using JPQL. Hibernate is the most popular JPA implementation. Spring Data JPA sits on top of JPA to further reduce boilerplate with repository interfaces.

■ Maven Dependencies

```
<!-- pom.xml -->
<dependency>
  <groupId>org.springframework.data</groupId>
  <artifactId>spring-data-jpa</artifactId>
  <version>3.2.0</version>
</dependency>
<dependency>
  <groupId>org.hibernate.orm</groupId>
  <artifactId>hibernate-core</artifactId>
  <version>6.4.0.Final</version>
</dependency>
<dependency>
  <groupId>com.h2database</groupId>
  <artifactId>h2</artifactId>
  <version>2.2.224</version>
</dependency>
```

5.2 Entity Mapping Annotations

Annotation	Description
@Entity	Marks class as a JPA entity (maps to a table)
@Table(name=...)	Specifies the table name (optional; defaults to class name)
@Id	Marks the primary key field
@GeneratedValue	Auto-generates primary key (IDENTITY, SEQUENCE, AUTO, TABLE)

@Column	Customises column mapping (name, nullable, length, unique)
@Transient	Field is NOT persisted to the database
@Temporal	Maps Date/Calendar fields to DATE, TIME or TIMESTAMP
@Enumerated	Maps an enum field to STRING or ORDINAL
@Lob	Maps large objects (CLOB, BLOB)
@OneToOne	One-to-one relationship between two entities
@OneToMany	One entity related to many entities (collection side)
@ManyToOne	Many entities related to one entity (FK side)
@ManyToMany	Many-to-many relationship
@JoinColumn	Specifies the foreign key column
@JoinTable	Specifies join table for @ManyToMany

■ Employee Entity

```
@Entity
@Table(name = "employees")
public class Employee {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(name = "first_name", nullable = false, length = 50)
    private String firstName;

    @Column(name = "last_name", nullable = false, length = 50)
    private String lastName;

    @Column(unique = true)
    private String email;

    private double salary;

    @Enumerated(EnumType.STRING)
    private Department department;

    // Constructors, getters, setters...
}
```

5.3 Entity Relationships

■ Entity Relationships

```
// @OneToMany / @ManyToOne example

@Entity
public class Department {
```

```

@Id @GeneratedValue(strategy = GenerationType.IDENTITY)
private Long id;
private String name;

@OneToMany(mappedBy = "department", cascade = CascadeType.ALL, fetch = FetchType.LAZY)
private List<Employee> employees = new ArrayList<>();
}

@Entity
public class Employee {
@Id @GeneratedValue(strategy = GenerationType.IDENTITY)
private Long id;
private String name;

@ManyToOne(fetch = FetchType.LAZY)
@JoinColumn(name = "dept_id") // FK column in employee table
private Department department;
}

// @ManyToMany example
@Entity
public class Student {
@Id @GeneratedValue(strategy = GenerationType.IDENTITY)
private Long id;

@ManyToMany
@JoinTable(name = "student_course",
joinColumns = @JoinColumn(name = "student_id"),
inverseJoinColumns = @JoinColumn(name = "course_id"))
private Set<Course> courses = new HashSet<>();
}

```

5.4 Spring Data JPA Repositories

Repository Interfaces

Spring Data JPA auto-generates implementations for repository interfaces at runtime. You just define an interface extending `CrudRepository`, `JpaRepository`, or `PagingAndSortingRepository` and Spring provides all CRUD operations with no implementation code.

■ EmployeeRepository – Spring Data JPA

```

// JpaRepository provides:
// save(), saveAll(), findById(), findAll(), count(), delete(), deleteById(), etc.
public interface EmployeeRepository extends JpaRepository<Employee, Long> {

// Derived query – Spring generates SQL from method name
List<Employee> findByLastName(String lastName);
List<Employee> findByDepartmentAndSalaryGreaterThan(String dept, double salary);
}

```

```

Optional<Employee> findByEmail(String email);
long countByDepartment(String department);
boolean existsByEmail(String email);
void deleteByEmail(String email);

// Custom JPQL query
@Query("SELECT e FROM Employee e WHERE e.salary BETWEEN :min AND :max")
List<Employee> findBySalaryRange(@Param("min") double min,
@Param("max") double max);

// Native SQL query
@Query(value = "SELECT * FROM employees WHERE dept=:dept LIMIT :n",
nativeQuery = true)
List<Employee> findTopNByDept(@Param("dept") String dept,
@Param("n") int n);

// Modifying query (UPDATE / DELETE)
@Modifying
@Transactional
@Query("UPDATE Employee e SET e.salary = e.salary * 1.1 WHERE e.department = :dept")
int raiseSalaries(@Param("dept") String dept);
}

```

■ JPA Configuration

```

// JPA Configuration (no Spring Boot)

@Configuration
@EnableJpaRepositories(basePackages = "com.example.repository")
@EnableTransactionManagement
public class JpaConfig {
    @Bean
    public DataSource dataSource() {
        // configure DataSource...
    }

    @Bean
    public LocalContainerEntityManagerFactoryBean entityManagerFactory(
        DataSource ds) {
        LocalContainerEntityManagerFactoryBean emf =
            new LocalContainerEntityManagerFactoryBean();
        emf.setDataSource(ds);
        emf.setPackagesToScan("com.example.model");
        HibernateJpaVendorAdapter adapter = new HibernateJpaVendorAdapter();
        adapter.setShowSql(true);
        adapter.setDatabasePlatform("org.hibernate.dialect.H2Dialect");
        emf.setJpaVendorAdapter(adapter);
    }
}

```

```

Properties props = new Properties();
props.put("hibernate.hbm2ddl.auto", "update"); // create/update schema
emf.setJpaProperties(props);
return emf;
}

@Bean
public PlatformTransactionManager transactionManager(
EntityManagerFactory emf) {
return new JpaTransactionManager(emf);
}
}

```

5.5 Service Layer with Transactions

■ EmployeeService – Transactional Service Layer

```

@Service
@Transactional // all public methods are transactional by default
public class EmployeeService {
    private final EmployeeRepository repo;

    public EmployeeService(EmployeeRepository repo) { this.repo = repo; }

    public Employee hire(Employee e) {
        return repo.save(e);
    }

    @Transactional(readOnly = true) // optimised for reads
    public List<Employee> getAllEmployees() {
        return repo.findAll();
    }

    @Transactional(readOnly = true)
    public Employee getEmployee(Long id) {
        return repo.findById(id)
            .orElseThrow(() -> new RuntimeException("Employee not found: " + id));
    }

    public void fire(Long id) {
        repo.deleteById(id);
    }

    public int giveRaise(String dept) {
        return repo.raiseSalaries(dept);
    }
}

```

5.6 Fetching Strategies & N+1 Problem

FetchType.LAZY vs FetchType.EAGER

LAZY loading defers loading of an association until it is accessed, improving performance. EAGER loading fetches the association immediately alongside the parent entity. Prefer LAZY and use JOIN FETCH in JPQL when you know you need the association.

■ JOIN FETCH to avoid N+1 problem

```
// Avoid N+1 with JOIN FETCH in JPQL
public interface DepartmentRepository extends JpaRepository<Department, Long> {
    @Query("SELECT d FROM Department d JOIN FETCH d.employees WHERE d.id = :id")
    Optional<Department> findByIdWithEmployees(@Param("id") Long id);
}
```


Quick Reference – Spring Ecosystem Summary

Module	Core Class/Interface	Key Annotation(s)	Purpose
Spring Core	ApplicationContext	@Component @Service @Autowired	IoC / DI container
Spring AOP	AspectJ / Proxy	@Aspect @Before @Around	Cross-cutting concerns
Spring JDBC	JdbcTemplate	@Repository @Transactional	SQL-level DB access
Spring JPA	JpaRepository	@Entity @Id @Query	ORM / Repository pattern
Maven	pom.xml / Lifecycle	N/A	Build & dependency mgmt
Eclipse	Workspace / Project	N/A	IDE / development env

■ This guide covers Spring 6.x and Spring Data JPA 3.x (Jakarta EE 10). For Spring Boot projects, most of this configuration is auto-configured – you only need application.properties and your repository/service/entity classes.